

untangle::async

Generated by Doxygen 1.18.0

1	async	1
2	Topic Index	5
2.1	Topics	5
3	Namespace Index	7
3.1	Namespace List	7
4	Class Index	9
4.1	Class List	9
5	File Index	11
5.1	File List	11
6	Topic Documentation	13
6.1	namespace untangle: functions	13
7	Namespace Documentation	15
7.1	untangle Namespace Reference	15
7.1.1	Detailed Description	15
8	Class Documentation	17
8.1	untangle::async::attachment Struct Reference	17
8.1.1	Detailed Description	17
8.2	untangle::async::execution< actionT > Class Template Reference	17
8.2.1	Detailed Description	19
8.2.2	Constructor & Destructor Documentation	19
8.2.2.1	execution() [1/2]	19
8.2.2.2	execution() [2/2]	19
8.2.2.3	~execution()	20
8.2.3	Member Function Documentation	20
8.2.3.1	create_instance()	20
8.2.3.2	bind_action_and_method()	21
8.2.3.3	bind_action_and_function()	21
8.2.3.4	is_running()	22
8.2.3.5	is_busy()	22
8.2.3.6	run()	22
8.2.3.7	start()	23
8.2.3.8	stop()	23
8.2.3.9	add_action()	23
8.2.3.10	attach()	24
8.2.3.11	detach()	24
8.2.3.12	results()	25
8.2.4	Member Data Documentation	25
8.2.4.1	action_execute	25
8.2.4.2	action_stop	26

8.2.4.3 <code>action_is_running</code>	26
8.2.4.4 <code>on_finished</code>	26
8.3 <code>untangle::async::execution_poll</code> Class Reference	26
8.3.1 Detailed Description	27
8.3.2 Member Function Documentation	27
8.3.2.1 <code>add()</code>	27
8.3.2.2 <code>remove()</code>	27
8.3.2.3 <code>is_running()</code>	28
8.3.2.4 <code>get()</code>	28
8.4 <code>untangle::async::invalid_attachment</code> Struct Reference	28
8.4.1 Detailed Description	28
8.4.2 Constructor & Destructor Documentation	28
8.4.2.1 <code>invalid_attachment()</code>	28
8.4.3 Member Function Documentation	29
8.4.3.1 <code>what()</code>	29
9 File Documentation	31
9.1 <code>async.hpp</code>	31
Index	37

Chapter 1

async

c++ mechanism to run queued callables in an asynchronous fashion

The interface consists of one header file that exposes a generic execution class. It is a wrapper of a list of callables that are invoked on other thread. The callables are pumped into the list by invoking an action from the caller thread.

The implementation is using the **actuator** callable, and therefore it exemplifies how a generic callable may improve the code structure, by easily creating interfaces inside a class instead of using external ones.

There are provided two execution modes: one-off and continuous.

requirements

C++23. The interface reports warnings with `std::println`, so it needs a standard library that provides `<print>` — GCC 14 or newer; older `libc++` and `MSVC` releases may not have it yet.

one-off execution



example: one-off execution of a plain function

```
#include <async.hpp>

void f()
{
    std::cout << "f thread " << std::this_thread::get_id() << std::endl;
}

int main()
{
    std::cout << "main thread " << std::this_thread::get_id() << std::endl;

    // declare an execution specialized for an action type; it must be held by a shared pointer,
    // because an async binding keeps only weak ownership of it
    // the name is optional, and identifies the execution in the warnings it reports
    using void_execution = untangle::async::execution<std::function<void(void)>>;
    auto execution = void_execution::create_instance("oneoff");
```

```

std::function<void(void)> action;
// create async binding between the action and f; the execution is passed as a shared pointer,
// which the binding holds weakly - that is what lets the action outlive it safely
void_execution::bind_action_and_function(action, f, execution);
// whenever the action is invoked, a new callable wrapping f will be added to the execution's action list
action();
// run the execution
execution->run();

// add the execution object to the running poll
untangle::async::execution_poll::get().add(*execution);

// wait the polled executions to finish
while(untangle::async::execution_poll::get().is_running())
{
    std::this_thread::sleep_for(std::chrono::milliseconds(1));
}

return 0;
}

```

example: one-off execution of a class method

```

#include <async.hpp>

class A
{
public:
    void f(int x)
    {
        std::cout << "A::f " << x << " thread " << std::this_thread::get_id() << std::endl;
    }
    std::function<void(int)> action;
};

int main()
{
    std::cout << "main thread " << std::this_thread::get_id() << std::endl;

    auto a = std::make_shared<A>();

    // declare an execution specialized for an action type; it must be held by a shared pointer
    using int_execution = untangle::async::execution<std::function<void(int)>>;
    auto execution = int_execution::create_instance("oneoff_method");
    // note both the bound object and the execution must be shared pointers
    int_execution::bind_action_and_method(a->action, a, &A::f, execution);
    // whenever the action is invoked, a new callable wrapping f will be added to the execution's action list
    a->action(10);
    // run the execution
    execution->run();

    // add the execution object to the running poll
    untangle::async::execution_poll::get().add(*execution);

    // wait the polled executions to finish
    while(untangle::async::execution_poll::get().is_running())
    {
        std::this_thread::sleep_for(std::chrono::milliseconds(1));
    }

    return 0;
}

```

continuous execution



example: continuous execution of a plain function

```

#include <async.hpp>

void f()
{
    std::cout << "f thread " << std::this_thread::get_id() << std::endl;
}

```

```

}

int main()
{
    std::cout << "main thread " << std::this_thread::get_id() << std::endl;

    // declare an execution specialized for an action type; it must be held by a shared pointer
    using void_execution = untangle::async::execution<std::function<void(void)>>;
    auto execution = void_execution::create_instance("continuous");
    std::function<void(void)> action;
    // create async binding between the action and f
    void_execution::bind_action_and_function(action, f, execution);

    // run the execution
    execution->start();
    // add f to the execution's queue (three times)
    action();action();action();
    // stop the execution
    execution->stop();

    // add the execution object to the running poll
    untangle::async::execution_poll::get().add(*execution);

    // wait the polled executions to finish
    while(untangle::async::execution_poll::get().is_running())
    {
        std::this_thread::sleep_for(std::chrono::milliseconds(1));
    }

    return 0;
}

```

queueing and lifetime

Queueing is thread safe. An action may be invoked from any thread; the execution's list is guarded, and the worker is woken rather than polled for.

`stop()` ends the execution's working life. The worker runs whatever was queued before the call and then leaves. An action invoked after `stop()` is refused rather than queued, because it would never run, and the refusal is reported:

```
warning: execution 'continuous' is stopped, action not added
```

An action bound to an object that has since been destroyed is dropped the same way, rather than ending the process:

```
warning: execution 'continuous' dropped an invalid action: bind: invalid object
```

The worker is detached and cannot be joined. Waiting for an execution therefore means waiting on its running state, which is what `execution_poll` reports and what the examples above do. The destructor waits on the same state, so an execution that goes out of scope while its worker is still running blocks until the worker is finished rather than leaving it reading freed memory.

The poll holds a pointer to each execution added to it, so an execution withdraws itself in its destructor. `execution_poll::remove()` is available for withdrawing one earlier; calling it is not required.

building the tests

```

cmake -S test -B test/build
cmake --build test/build
ctest --test-dir test/build

```

`test/CMakeLists.txt` fetches googletest at configure time, so the first configure needs network access. The actuator submodule has to be present: `git submodule update --init`.

sanitizers

ASYNC_SANITIZE builds the tests under a sanitizer. It is off by default, because a sanitized build is several times slower and ThreadSanitizer does not ship for every toolchain.

```
cmake -S test -B test/build-asan -DASYNC_SANITIZE=address
cmake --build test/build-asan
ctest --test-dir test/build-asan
```

Accepted values are address, thread, undefined, or empty. Anything else is refused at configure time rather than passed through to the compiler.

Use a separate build directory per sanitizer. address and thread instrument the same accesses in incompatible ways and cannot be combined, so one build directory is one sanitizer.

The two answer different questions. ThreadSanitizer names both sides of a data race, which is how the queue's races were diagnosed. AddressSanitizer reports a use-after-free, which is how the dangling attachment was. Neither sees everything: a defect reached through a destroyed `std::mutex` throws `std::system_error` from inside `libsystem` before any instrumented load runs, so both sanitizers stay silent. Reach for a direct instrumented read when that happens.

Chapter 2

Topic Index

2.1 Topics

Here is a list of all topics with brief descriptions:

namespace untangle: functions [13](#)

Chapter 3

Namespace Index

3.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

untangle	Interface to untangle::async::execution class	15
--------------------------	---	--------------------

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

untangle::async::attachment	
One execution's place in the attachment graph: who attached it, if anyone	17
untangle::async::execution< actionT >	
Asynchronous template execution class	17
untangle::async::execution_poll	
Single-tone poll that reports whether any execution added to it is still running	26
untangle::async::invalid_attachment	
Raised by execution::attach() when the attachment asked for is refused	28

Chapter 5

File Index

5.1 File List

Here is a list of all documented files with brief descriptions:

async.hpp		31
---------------------------	-------	----

Chapter 6

Topic Documentation

6.1 namespace untangle: functions

Chapter 7

Namespace Documentation

7.1 untangle Namespace Reference

Interface to [untangle::async::execution](#) class.

7.1.1 Detailed Description

Interface to [untangle::async::execution](#) class.

Chapter 8

Class Documentation

8.1 `untangle::async::attachment` Struct Reference

One execution's place in the attachment graph: who attached it, if anyone.

```
#include <async.hpp>
```

Public Attributes

- `std::weak_ptr< attachment >` `attacher`
The link of the execution that attached this one; empty when nothing has.

8.1.1 Detailed Description

One execution's place in the attachment graph: who attached it, if anyone.

At namespace scope, so it is one type rather than one per specialisation: an execution may be attached by an execution of any specialisation, and the chain is walked without knowing any link's action type. It dies with its execution, so it doubles as a liveness token - a link that still locks belongs to an execution that is still there.

The documentation for this struct was generated from the following file:

- `async.hpp`

8.2 `untangle::async::execution< actionT >` Class Template Reference

Asynchronous template execution class.

```
#include <async.hpp>
```

Public Member Functions

- [execution](#) ()
Constructs an execution with the default name. Call [create_instance\(\)](#) instead.
- [execution](#) (std::string exec_name)
Constructs a named execution. Use [create_instance\(\)](#) for one that is to be bound.
- [~execution](#) ()
Destroys the execution, once its worker has left.
- bool [is_running](#) () const
Is this execution still working?
- bool [is_busy](#) () const
Is this execution working through its action list?
- void [run](#) ()
Runs what is queued on a worker of its own, once, and keeps the results.
- void [start](#) ()
Starts a continuous worker that runs whatever is queued until [stop\(\)](#).
- void [stop](#) ()
Stops the worker, after the actions already queued have run.
- template<typename... Args>
bool [add_action](#) (actionT action, Args... args)
Queues an action, bound to args, and says whether it was taken.
- template<typename otherT>
void [attach](#) (const std::shared_ptr< otherT > &other)
Attaches another execution, so that this one triggers it.
- template<typename otherT>
bool [detach](#) (otherT &other)
Detaches an execution previously attached by [attach\(\)](#).
- auto [results](#) () const
The return values of the actions run by [run\(\)](#), in the order they ran.

Static Public Member Functions

- template<typename... Args>
static std::shared_ptr< execution > [create_instance](#) (Args &&... args)
Creates an execution owned by a std::shared_ptr, which is what binding to it requires.
- template<typename classT, typename T>
static void [bind_action_and_method](#) (actionT &action, const std::shared_ptr< classT > &obj, T classT::*method, const std::shared_ptr< execution > &async_exec)
Binds asynchronously an external action to a class function member.
- template<typename T>
static void [bind_action_and_function](#) (actionT &action, const T &Fn, const std::shared_ptr< execution > &async_exec)
Binds asynchronously an external action to a plain function.

Public Attributes

- std::function< void(void)> [action_execute](#)
Connection point that runs this execution's queued actions. Bound to [execute_actions\(\)](#).
- std::function< void(void)> [action_stop](#)
Connection point that stops this execution. Bound to [stop\(\)](#).
- std::function< bool(void)> [action_is_running](#)
Connection point that reports whether this execution is running. Bound to [is_running\(\)](#).
- std::function< void(void)> [on_finished](#)
Called on the worker's thread once the action list has drained. Assigned by the caller.

Static Public Attributes

- `static constexpr auto default_name = "default_name"`
Name of an execution built unnamed.

8.2.1 Detailed Description

```
template<typename actionT>
class untangle::async::execution< actionT >
```

Asynchronous template execution class.

An async execution provides a mechanism to queue actions and execute them sequentially on a separate thread. It may also attach another [execution](#) object and trigger its actions. This way actions of different types may be executed on the same thread.

The mechanism relies on an "asynchronous binding" created by [bind_action_and_method\(\)](#) or [bind_action_and_function\(\)](#).

Template Parameters

<code>actionT</code>	It represents the type of the action. It is specified as <code>std::function<...></code> and should match the signature of the bound function or class method.
----------------------	--

8.2.2 Constructor & Destructor Documentation

8.2.2.1 `execution()` [1/2]

```
template<typename actionT>
untangle::async::execution< actionT >::execution () [inline]
```

Constructs an execution with the default name. Call [create_instance\(\)](#) instead.

The name identifies this execution in the warnings it reports, and is otherwise unused.

8.2.2.2 `execution()` [2/2]

```
template<typename actionT>
untangle::async::execution< actionT >::execution (
    std::string exec_name) [inline], [explicit]
```

Constructs a named execution. Use [create_instance\(\)](#) for one that is to be bound.

Parameters

exec_name	- A name for this execution.
-----------	------------------------------

8.2.2.3 ~execution()

```
template<typename actionT>
untangle::async::execution< actionT >::~~execution () [inline]
```

Destroys the execution, once its worker has left.

The worker is detached and cannot be joined, so this waits on the "running" state instead. It detaches this execution from whatever attached it first, and returns at once for one that was never started.

Attention

Safe only because running_ is cleared as the very last thing the worker does: this object may be freed the moment it reads false, so nothing may follow it in execute() or loop().

Remarks

No rule of five, deliberately. An execution is declared once and used in place; the members already delete the copy operations several times over.

8.2.3 Member Function Documentation

8.2.3.1 create_instance()

```
template<typename actionT>
template<typename... Args>
std::shared_ptr< execution > untangle::async::execution< actionT >::create_instance (
    Args &&... args) [inline], [static]
```

Creates an execution owned by a std::shared_ptr, which is what binding to it requires.

An execution built any other way is still usable - [add_action\(\)](#) takes actions directly - but cannot be bound, and the attempt does not compile.

Parameters

args	- Constructor arguments: a name, or nothing for the default name.
------	---

Returns

- A std::shared_ptr owning the new execution.

8.2.3.2 bind_action_and_method()

```
template<typename actionT>
template<typename classT, typename T>
void untangle::async::execution< actionT >::bind_action_and_method (
    actionT & action,
    const std::shared_ptr< classT > & obj,
    T classT::* method,
    const std::shared_ptr< execution< actionT > > & async_exec) [inline], [static]
```

Binds asynchronously an external action to a class function member.

It creates an action as a binding to a class method (by untangle::bind()), and assigns to action a callable that passes it to [add_action\(\)](#). The execution is held weakly, so the action may outlive it.

Attention

Invoking action after the execution has been destroyed throws untangle::invalid_action. The worker catches it and drops the action with a warning.

Parameters

action	[in,out] - An action of type std::function<...>.
obj	- A std::shared_ptr that wraps the bound class object.
method	- Pointer to function member. It is specified as &<class type>::<function member>.
async_exec	- A std::shared_ptr owning the execution the action is queued on. Requiring it here is what keeps an execution that no shared_ptr owns from being bound at all.

8.2.3.3 bind_action_and_function()

```
template<typename actionT>
template<typename T>
void untangle::async::execution< actionT >::bind_action_and_function (
    actionT & action,
    const T & Fn,
    const std::shared_ptr< execution< actionT > > & async_exec) [inline], [static]
```

Binds asynchronously an external action to a plain function.

It wraps Fn in an action and assigns to action a callable that passes it to [add_action\(\)](#). The execution is held weakly, as in [bind_action_and_method\(\)](#).

Parameters

action	[in,out] - An action of type std::function<...>.
Fn	- A plain function.
async_exec	- A std::shared_ptr owning the execution the action is queued on.

8.2.3.4 `is_running()`

```
template<typename actionT>
bool untangle::async::execution< actionT >::is_running () const [inline]
```

Is this execution still working?

False from the moment the worker starts reporting itself finished, so a caller polling it learns a run is over without waiting for the object to become safe to destroy.

Remarks

[on_finished](#) is raised from inside the drain and reads this as true - the worker is standing in the callback with the rest of its path still to go.

Returns

true - the execution is still working.

8.2.3.5 `is_busy()`

```
template<typename actionT>
bool untangle::async::execution< actionT >::is_busy () const [inline]
```

Is this execution working through its action list?

True from the moment an action is added until the last one has returned. Unlike [is_running\(\)](#), which is about the worker thread, this answers "has it room for more work"

- a worker started by [start\(\)](#) is running for its whole life whether busy or idle.

Remarks

Answers for this execution's own actions, not for those it triggers through [attach\(\)](#).

A false answer is durable only for the caller that is itself adding the actions; a true one may stop being true the moment after.

Returns

true - actions are queued, or one is running.

8.2.3.6 `run()`

```
template<typename actionT>
void untangle::async::execution< actionT >::run () [inline]
```

Runs what is queued on a worker of its own, once, and keeps the results.

The worker is detached: wait for it with [is_running\(\)](#) or [execution_poll](#). Actions queued while it is draining are run by it too. [results\(\)](#) is filled by this path only.

8.2.3.7 start()

```
template<typename actionT>
void untangle::async::execution< actionT >::start () [inline]
```

Starts a continuous worker that runs whatever is queued until `stop()`.

Unlike `run()`, the worker stays for the life of the execution, so `is_running()` is true whether or not there is work; ask `is_busy()` instead. Results are not collected.

8.2.3.8 stop()

```
template<typename actionT>
void untangle::async::execution< actionT >::stop () [inline]
```

Stops the worker, after the actions already queued have run.

Remarks

An action passed to `add_action()` after this call is refused, not queued: `stop()` is the end of this execution's working life, and an action accepted then would never run.

8.2.3.9 add_action()

```
template<typename actionT>
template<typename... Args>
bool untangle::async::execution< actionT >::add_action (
    actionT action,
    Args... args) [inline]
```

Queues an action, bound to args, and says whether it was taken.

It runs later, on whichever thread drives this execution - its own worker, or an attacher's. Queueing before `run()` or `start()` is normal.

Attention

A stopped execution refuses and drops the action, silently for a caller that ignores the answer. Nothing is thrown: `untangle::invalid_action` means a dead binding, while a refused action is sound and merely too late.

Remarks

A caller arriving through `bind_action_and_method()` or `bind_action_and_function()` never sees the answer: those lambdas return `actionT::result_type`, which has no room for it.

Template Parameters

Args	- The argument types the action is bound to.
------	--

Parameters

action	- The action to queue.
args	- The arguments to bind to action.

Returns

- true - queued, and this execution's worker will run it.
- false - the execution is stopped and the action was dropped.

8.2.3.10 attach()

```
template<typename actionT>
template<typename otherT>
void untangle::async::execution< actionT >::attach (
    const std::shared_ptr< otherT > & other) [inline]
```

Attaches another execution, so that this one triggers it.

The attached execution's pending actions are run by this one's worker, and stopping this one stops that one too. Actions of different types can be run on one thread this way.

The attachment is a pointer into other; what makes that safe is the record left on the other side, which [~execution\(\)](#) and [detach\(\)](#) use to take it back out.

Parameters

other	- A std::shared_ptr owning the execution to trigger; requiring one keeps an unowned execution from being attached at all. Attaching the same execution twice adds it twice, and one detach() still removes it completely.
-------	---

Exceptions

invalid_attachment	- if other is attached already. An execution has at most one attacher, which keeps the graph a forest and the cycle check a walk rather than a search.
invalid_attachment	- if the attachment would close a cycle, itself included. Driving a cycle recurses until the stack is gone, so it is refused where the caller can still be told.

8.2.3.11 detach()

```
template<typename actionT>
template<typename otherT>
bool untangle::async::execution< actionT >::detach (
    otherT & other) [inline]
```

Detaches an execution previously attached by [attach\(\)](#).

Both paths are unwired together, the triggering one and the stopping one.

Parameters

other	- The execution to stop triggering. Detaching one that was never attached is not an error: the caller asked for a state that already holds.
-------	---

Returns

- true - other was attached and is no longer.
- false - other was not attached to this execution.

8.2.3.12 results()

```
template<typename actionT>
auto untangle::async::execution< actionT >::results () const [inline]
```

The return values of the actions run by [run\(\)](#), in the order they ran.

Read it from [on_finished](#), which fires once the queue has drained, so the results are complete by the time the callback sees them. Returned by value: the worker is appending to it.

Remarks

Only [run\(\)](#) fills this. A continuous worker has no point at which a run is over, so [start\(\)](#) collects nothing.

Returns

The results of the most recent run, or empty if none has produced any.

8.2.4 Member Data Documentation

8.2.4.1 action_execute

```
template<typename actionT>
std::function<void(void)> untangle::async::execution< actionT >::action_execute
```

Connection point that runs this execution's queued actions. Bound to [execute_actions\(\)](#).

Attention

Wired by the constructor. Assigning to it unwires whatever is connected, silently.

8.2.4.2 `action_stop`

```
template<typename actionT>
std::function<void(void)> untangle::async::execution< actionT >::action_stop
```

Connection point that stops this execution. Bound to [stop\(\)](#).

[attach\(\)](#) wires it together with [action_execute](#), and [detach\(\)](#) unwires both.

Attention

Wired by the constructor. Assigning to it unwires whatever is connected, silently.

8.2.4.3 `action_is_running`

```
template<typename actionT>
std::function<bool(void)> untangle::async::execution< actionT >::action_is_running
```

Connection point that reports whether this execution is running. Bound to [is_running\(\)](#).

[execution_poll](#) holds its address, and [~execution\(\)](#) takes it back out.

Attention

Wired by the constructor. Assigning to it unwires whatever is connected, silently.

8.2.4.4 `on_finished`

```
template<typename actionT>
std::function<void(void)> untangle::async::execution< actionT >::on_finished
```

Called on the worker's thread once the action list has drained. Assigned by the caller.

[results\(\)](#) is complete by the time it runs, and [is_running\(\)](#) reads true there.

The documentation for this class was generated from the following file:

- `async.hpp`

8.3 `untangle::async::execution_poll` Class Reference

Single-tone poll that reports whether any [execution](#) added to it is still running.

```
#include <async.hpp>
```

Public Member Functions

- `template<typename asyncexecT>`
`void add (asyncexecT &async_exec)`
 Adds an [execution](#) object to the poll.
- `template<typename asyncexecT>`
`void remove (asyncexecT &async_exec)`
 Removes an [execution](#) object from the poll.
- `auto is_running ()`
 Checks if the execution objects are running.

Static Public Member Functions

- `static execution_poll & get ()`
 Gets the single-tone instance.

8.3.1 Detailed Description

Single-tone poll that reports whether any [execution](#) added to it is still running.

Remarks

An execution runs in a detached thread and cannot be joined, so checking the "running" state is the only way to wait for one. This checks any number of them at once.

8.3.2 Member Function Documentation

8.3.2.1 add()

```
template<typename asyncexecT>
void untangle::async::execution_poll::add (
    asyncexecT & async_exec) [inline]
```

Adds an [execution](#) object to the poll.

Parameters

<code>async_exec</code>	An execution object.
-------------------------	--------------------------------------

8.3.2.2 remove()

```
template<typename asyncexecT>
void untangle::async::execution_poll::remove (
    asyncexecT & async_exec) [inline]
```

Removes an [execution](#) object from the poll.

Generated by Doxygen

Attention

[add\(\)](#) stores a pointer into the object, so one destroyed while still added would leave the poll calling

Parameters

async_exec	An execution object.
------------	--------------------------------------

8.3.2.3 is_running()

```
auto untangle::async::execution_poll::is_running () [inline]
```

Checks if the execution objects are running.

Returns

true - if at least one [execution](#) object in this poll is running.

8.3.2.4 get()

```
execution_poll & untangle::async::execution_poll::get () [inline], [static]
```

Gets the single-tone instance.

Returns

[execution_poll](#) An [execution_poll](#) instance.

The documentation for this class was generated from the following file:

- [async.hpp](#)

8.4 untangle::async::invalid_attachment Struct Reference

Raised by [execution::attach\(\)](#) when the attachment asked for is refused.

```
#include <async.hpp>
```

Public Member Functions

- [invalid_attachment](#) (std::string text)
Construct a new invalid attachment object.
- const char * [what](#) () const noexcept override
The message text describing the reason of this exception.

8.4.1 Detailed Description

Raised by [execution::attach\(\)](#) when the attachment asked for is refused.

Generated by Doxygen

Remarks

Not an untangle::invalid_action: that one reports a binding whose target has died and is swallowed by the kernel, while this one reports a binding whose target has died and is not swallowed.

Parameters

text	- A message text, describing the reason of this exception.
------	--

8.4.3 Member Function Documentation

8.4.3.1 what()

```
const char * untangle::async::invalid_attachment::what () const    [inline], [override], [noexcept]
```

The message text describing the reason of this exception.

Returns

const char* - The message text.

The documentation for this struct was generated from the following file:

- async.hpp

Chapter 9

File Documentation

9.1 async.hpp

```
00001 // Copyright (c) 2018 Nicolae Popescu. MIT License.
00002
00006 #pragma once
00007
00008 #include <actuator/actuator.hpp>
00009 #include <algorithm>
00010 #include <atomic>
00011 #include <condition_variable>
00012 #include <exception>
00013 #include <memory>
00014 #include <mutex>
00015 #include <print>
00016 #include <string>
00017 #include <thread>
00018 #include <vector>
00019
00020 namespace untangle {
00021 template <typename actionT>
00022 struct actuator;
00023
00024 namespace async {
00025 template <typename actionT>
00026 class execution;
00027 }
00028 } // namespace untangle
00029
00030 namespace untangle {
00031 namespace async {
00035
00042 struct invalid_attachment : std::exception {
00048     explicit invalid_attachment(std::string text) : message_(std::move(text)) {}
00049
00055     const char* what() const noexcept override { return message_.c_str(); }
00056
00057 private:
00058     std::string message_;
00059 };
00060
00069 struct attachment {
00071     std::weak_ptr<attachment> attacher;
00072 };
00073
00080 class execution_poll {
00081 public:
00087     template <typename asyncexecT>
00088     void add(asyncexecT& async_exec) {
00089         std::lock_guard<std::mutex> lock(actuator_mutex_);
00090
00091         if (!actuator_is_running_.is_connected()) {
00092             actuator_is_running_ = untangle::connect(async_exec.action_is_running);
00093         } else {
00094             actuator_is_running_.add(&async_exec.action_is_running);
00095         }
00096     }
00097
00106     template <typename asyncexecT>
00107     void remove(asyncexecT& async_exec) {
```

```

00108     std::lock_guard<std::mutex> lock(actuator_mutex_);
00109
00110     actuator_is_running_.remove(&async_exec.action_is_running);
00111 }
00112
00113 auto is_running() {
00114     std::lock_guard<std::mutex> lock(actuator_mutex_);
00115     actuator_is_running_();
00116
00117     auto result = false;
00118     for (const auto& ret : actuator_is_running_.results) {
00119         result |= ret;
00120     }
00121     return result;
00122 }
00123
00124 static execution_poll& get() {
00125     static execution_poll instance;
00126     return instance;
00127 }
00128
00129 private:
00130     execution_poll() = default;
00131     ~execution_poll() = default;
00132     actuator<std::function<bool(void)>> actuator_is_running_;
00133     mutable std::mutex actuator_mutex_;
00134 };
00135
00136 template <typename actionT>
00137 class execution {
00138     // An execution may attach one of a different specialisation, which is a different class;
00139     // attach() and detach() reach into its attachment_lifetime_.
00140     template <typename otherActionT>
00141     friend class execution;
00142
00143 public:
00144     template <typename... Args>
00145     static std::shared_ptr<execution> create_instance(Args&&... args) {
00146         return std::make_shared<execution>(std::forward<Args>(args)...);
00147     }
00148
00149     execution() : execution(std::string(default_name)) {}
00150
00151     explicit execution(std::string exec_name) : name(std::move(exec_name)) {
00152         other_this_ = this;
00153         action_execute = untangle::bind(other_this_, &execution<actionT>::execute_actions);
00154         action_stop = untangle::bind(other_this_, &execution<actionT>::stop);
00155         action_is_running = untangle::bind(other_this_, &execution<actionT>::is_running);
00156     }
00157
00158     ~execution() {
00159         // Out of the attacher first, before the worker is even asked to stop: from here on nothing
00160         // may reach this object, and the actions about to be destroyed are the ones it holds.
00161         if (!attachment_lifetime_->attacher.expired()) {
00162             attacher_execute->remove(&action_execute);
00163             attacher_stop->remove(&action_stop);
00164         }
00165
00166         {
00167             std::lock_guard<std::mutex> lock(action_mutex_);
00168             started_ = false;
00169             stopped_ = true;
00170         }
00171
00172         action_cv_.notify_all();
00173
00174         while (running_.load()) { // time of check
00175             std::this_thread::sleep_for(std::chrono::milliseconds(1));
00176         }
00177
00178         // time of use: this object is freed once the destructor returns, so the check above is only
00179         // safe because running_ is the last thing the worker touches
00180         execution_poll::get().remove(*this);
00181     }
00182
00183     template <typename classT, typename T>
00184     static void bind_action_and_method(actionT& action, const std::shared_ptr<classT>& obj,
00185                                       T classT::* method,
00186                                       const std::shared_ptr<execution>& async_exec) {
00187         actionT async_action = untangle::bind(obj, method);
00188
00189         action = [wp = std::weak_ptr<execution>(async_exec),
00190                  async_action](auto... args) -> actionT::result_type {
00191             // lock() also keeps the execution alive for the duration of the call
00192             const auto exec = wp.lock();
00193             if (!exec) {
00194                 throw invalid_action("bind: invalid execution");
00195             }
00196         };
00197     }

```

```

00268     }
00269     exec->add_action(async_action, args...);
00270     return typename actionT::result_type();
00271 };
00272 }
00273
00284 template <typename T>
00285 static void bind_action_and_function(actionT& action, const T& Fn,
00286                                     const std::shared_ptr<execution>& async_exec) {
00287     actionT async_action = Fn;
00288
00289     action = [wp = std::weak_ptr<execution>(async_exec),
00290             async_action](auto... args) -> actionT::result_type {
00291         const auto exec = wp.lock();
00292         if (!exec) {
00293             throw invalid_action("bind: invalid execution");
00294         }
00295         exec->add_action(async_action, args...);
00296         return typename actionT::result_type();
00297     };
00298 }
00299
00311 bool is_running() const { return running_.load() && !finishing_.load(); }
00312
00328 bool is_busy() const {
00329     std::lock_guard<std::mutex> lock(action_mutex_);
00330     return !action_list_.empty() || executing_action_.load();
00331 }
00332
00339 void run() {
00340     running_ = true;
00341     finishing_ = false;
00342     collecting_results_ = true;
00343     this_thread_ = std::thread(&execution::execute, this);
00344     this_thread_.detach();
00345 }
00346
00353 void start() {
00354     running_ = true;
00355     finishing_ = false;
00356     collecting_results_ = false;
00357     {
00358         std::lock_guard<std::mutex> lock(action_mutex_);
00359         started_ = true;
00360         stopped_ = false;
00361     }
00362
00363     this_thread_ = std::thread(&execution::loop, this);
00364     this_thread_.detach();
00365 }
00366
00373 void stop() {
00374     {
00375         std::lock_guard<std::mutex> lock(action_mutex_);
00376         started_ = false;
00377         stopped_ = true;
00378     }
00379
00380     // The worker drains what is already queued before it leaves loop(), so stop() does not have to
00381     // spin on the list; waking it is enough.
00382     action_cv_.notify_all();
00383
00384     if (actuator_stop_.is_connected()) {
00385         actuator_stop_();
00386     }
00387 }
00388
00410 template <typename... Args>
00411 bool add_action(actionT action, Args... args) {
00412     {
00413         std::lock_guard<std::mutex> lock(action_mutex_);
00414
00415         // Once stopped, the worker is on its way out and would never reach this action; dropping it
00416         // here is what keeps it from sitting in the list looking as though it were pending.
00417         if (stopped_) {
00418             std::println(stderr, "warning: execution '{}' is stopped, action not added", name);
00419             return false;
00420         }
00421
00422         action_list_.push_back(std::bind(action, args...));
00423     }
00424
00425     action_cv_.notify_one();
00426     return true;
00427 }
00428
00448 template <typename otherT>

```

```

00449 void attach(const std::shared_ptr<otherT>& other) {
00450     if (static_cast<const void*>(other.get()) == static_cast<const void*>(this)) {
00451         throw invalid_attachment("attach: an execution cannot be attached to itself");
00452     }
00453
00454     if (!other->attachment_lifetime_->attacher.expired()) {
00455         throw invalid_attachment("attach: that execution is attached already");
00456     }
00457
00458     // At most one attacher each makes the graph a forest, so a cycle closes exactly when other is
00459     // somewhere up this chain. Walking it is the whole check.
00460     for (auto link = attachment_lifetime_->attacher.lock(); link; link = link->attacher.lock()) {
00461         if (link == other->attachment_lifetime_) {
00462             throw invalid_attachment("attach: the attachment would close a cycle");
00463         }
00464     }
00465
00466     if (!actuator_execute_.is_connected()) {
00467         actuator_execute_ = untangle::connect(other->action_execute);
00468     } else {
00469         actuator_execute_.add(&other->action_execute);
00470     }
00471
00472     if (!actuator_stop_.is_connected()) {
00473         actuator_stop_ = untangle::connect(other->action_stop);
00474     } else {
00475         actuator_stop_.add(&other->action_stop);
00476     }
00477
00478     // What ~execution() needs to take itself back out of these actuators: the link proves this
00479     // attacher is still there, and the two pointers are only ever followed while it does.
00480     other->attachment_lifetime_->attacher = attachment_lifetime_;
00481     other->attacher_execute_ = &actuator_execute_;
00482     other->attacher_stop_ = &actuator_stop_;
00483 }
00484
00496 template <typename otherT>
00497 bool detach(otherT& other) {
00498     // The actuator's own action list is the record of what is attached; nothing else has to keep
00499     // one. An action is stored as a pointer, so the attachment is found by identity.
00500     const auto& actions = actuator_execute_.actions;
00501     if (std::find(actions.begin(), actions.end(), &other.action_execute) == actions.end()) {
00502         return false;
00503     }
00504
00505     actuator_execute_.remove(&other.action_execute);
00506     actuator_stop_.remove(&other.action_stop);
00507
00508     other.attachment_lifetime_->attacher.reset();
00509     other.attacher_execute_ = nullptr;
00510     other.attacher_stop_ = nullptr;
00511
00512     return true;
00513 }
00514
00526 auto results() const {
00527     std::lock_guard<std::mutex> lock(results_mutex_);
00528     return results_;
00529 }
00530
00531 static constexpr auto default_name = "default_name";
00532
00538 std::function<void(void)> action_execute;
00539
00547 std::function<void(void)> action_stop;
00548
00557 std::function<bool(void)> action_is_running;
00558
00564 std::function<void(void)> on_finished;
00565 std::string name = default_name;
00566
00567 private:
00568     using queued_action_t = std::function<typename actionT::result_type(void)>;
00569
00576 void execute_action(queued_action_t& action) {
00577     if constexpr (std::is_void_v<typename actionT::result_type>) {
00578         action();
00579     } else {
00580         auto value = action();
00581
00582         if (collecting_results_) {
00583             std::lock_guard<std::mutex> lock(results_mutex_);
00584             results_.push_back(std::move(value));
00585         }
00586     }
00587 }
00588

```

```

00598 void execute_actions() {
00599     actions_run_ = 0;
00600
00601     // The action is taken off the list under the lock and invoked with the lock released: an action
00602     // is caller code that may run for a while, and may itself call add_action().
00603     for (;;) {
00604         queued_action_t action;
00605
00606         {
00607             std::lock_guard<std::mutex> lock(action_mutex_);
00608             if (action_list_.empty()) {
00609                 break;
00610             }
00611
00612             action = std::move(action_list_.front());
00613             action_list_.pop_front();
00614
00615             // Under the lock that emptied the list, so is_busy() sees the pop and this together or
00616             // neither.
00617             executing_action_ = true;
00618         }
00619
00620         // Nothing may leave this loop: an exception escaping a detached thread function calls
00621         // std::terminate.
00622         try {
00623             execute_action(action);
00624         } catch (const invalid_action& ia) {
00625             std::println(stderr, "warning: execution '{}' dropped an invalid action: {}", name,
00626                 ia.what());
00627         } catch (const std::exception& e) {
00628             std::println(stderr, "warning: execution '{}' dropped an action that threw: {}", name,
00629                 e.what());
00630         } catch (...) {
00631             std::println(stderr, "warning: execution '{}' dropped an action that threw an unknown type",
00632                 name);
00633         }
00634
00635         executing_action_ = false;
00636
00637         // Counted whether it ran cleanly, reported a dead binding, or threw: it came off the queue,
00638         // and the queue is what the notification is about.
00639         ++actions_run_;
00640
00641         // The notification is raised without the lock: on_finished may call add_action(), which
00642         // takes it.
00643         bool drained = false;
00644         {
00645             std::lock_guard<std::mutex> lock(action_mutex_);
00646             drained = action_list_.empty();
00647         }
00648
00649         if (drained) {
00650             notify_finished();
00651         }
00652     }
00653
00654     // Last, and part of the pass: an attachment is only ever reached through action_execute, which
00655     // is this function, so whoever drives this execution drives the ones attached to it.
00656     if (actuator_execute_.is_connected()) {
00657         actuator_execute_();
00658     }
00659 }
00660
00661 void notify_finished() {
00662     if (actions_run_ == 0 || !on_finished) {
00663         return;
00664     }
00665
00666     {
00667         // An action may queue another, so a pass that drained can leave more behind it. That is the
00668         // next batch, not the end of this one.
00669         std::lock_guard<std::mutex> lock(action_mutex_);
00670         if (!action_list_.empty()) {
00671             return;
00672         }
00673     }
00674
00675     on_finished();
00676 }
00677
00678 void execute() {
00679     // The results belong to this run, not to this run and the last one.
00680     {
00681         std::lock_guard<std::mutex> lock(results_mutex_);
00682         results_.clear();
00683     }
00684 }

```

```

00689     execute_actions();
00690
00691     // After the drain and the callback it raised: there is no more work to be told about.
00692     finishing_ = true;
00693
00694     std::println("execution '{}'" finishing thread", name);
00695
00696     // Must stay last: ~execution() may free this object the moment it reads false.
00697     running_ = false;
00698 }
00699
00700 void loop() {
00701     for (;;) {
00702         {
00703             std::unique_lock<std::mutex> lock(action_mutex_);
00704
00705             // Bounded: an attached execution has its own list and cannot notify this condition
00706             // variable.
00707             action_cv_.wait_for(lock, std::chrono::milliseconds(10),
00708                 [this] { return !action_list_.empty() || !started_; });
00709
00710             if (!started_ && action_list_.empty()) {
00711                 break;
00712             }
00713         }
00714         execute_actions();
00715     }
00716
00717     // A last pass for the attachments; this execution's own list is already empty here.
00718     execute_actions();
00719
00720     // After the last pass, because that pass still runs work.
00721     finishing_ = true;
00722
00723     std::println("execution '{}'" thread finished", name);
00724
00725     // Must stay last: ~execution() may free this object the moment it reads false.
00726     running_ = false;
00727 }
00728
00729 std::thread this_thread_;
00730
00731 std::atomic_bool started_ = {false};
00732 std::atomic_bool running_ = {false};
00733
00734 // Set by stop(), and the difference between "not started yet" and "finished for good": actions
00735 // may be queued before run()/start(), but not after stop().
00736 std::atomic_bool stopped_ = {false};
00737
00738 // action_list_, started_ and stopped_ are written by every thread that calls add_action() or
00739 // stop() and read by the worker; nothing touches them outside this mutex.
00740 mutable std::mutex action_mutex_;
00741 std::condition_variable action_cv_;
00742
00743 std::list<std::function<typename actionT::result_type(void)>> action_list_;
00744
00752 using void_actuator = actuator<std::function<void(void)>>;
00753
00754 void_actuator actuator_execute_;
00755 void_actuator actuator_stop_;
00756
00768 std::shared_ptr<attachment> attachment_lifetime_ = std::make_shared<attachment>();
00769
00781 void_actuator* attacher_execute_ = nullptr;
00782 void_actuator* attacher_stop_ = nullptr;
00783
00784 // std::vector cannot hold void type; use an arbitrary type e.g. int
00785 using resultT = std::conditional<std::is_void<typename actionT::result_type>::value, int,
00786     typename actionT::result_type>;
00787
00791 std::vector<typename resultT::type> results_;
00792
00796 mutable std::mutex results_mutex_;
00797
00804 std::atomic_bool collecting_results_ = {false};
00805
00816 std::atomic_bool finishing_ = {false};
00817
00825 std::atomic_bool executing_action_ = {false};
00826
00834 std::atomic_size_t actions_run_ = {0};
00835
00836 execution* other_this_;
00837 };
00838 } // namespace async
00839 } // namespace untangle

```

Index

- ~execution
 - untangle::async::execution< actionT >, [20](#)
- action_execute
 - untangle::async::execution< actionT >, [25](#)
- action_is_running
 - untangle::async::execution< actionT >, [26](#)
- action_stop
 - untangle::async::execution< actionT >, [25](#)
- add
 - untangle::async::execution_poll, [27](#)
- add_action
 - untangle::async::execution< actionT >, [23](#)
- async, [1](#)
- attach
 - untangle::async::execution< actionT >, [24](#)
- bind_action_and_function
 - untangle::async::execution< actionT >, [21](#)
- bind_action_and_method
 - untangle::async::execution< actionT >, [20](#)
- create_instance
 - untangle::async::execution< actionT >, [20](#)
- detach
 - untangle::async::execution< actionT >, [24](#)
- execution
 - untangle::async::execution< actionT >, [19](#)
- get
 - untangle::async::execution_poll, [28](#)
- invalid_attachment
 - untangle::async::invalid_attachment, [28](#)
- is_busy
 - untangle::async::execution< actionT >, [22](#)
- is_running
 - untangle::async::execution< actionT >, [21](#)
 - untangle::async::execution_poll, [28](#)
- namespace untangle: functions, [13](#)
- on_finished
 - untangle::async::execution< actionT >, [26](#)
- remove
 - untangle::async::execution_poll, [27](#)
- results
 - untangle::async::execution< actionT >, [25](#)
- run
 - untangle::async::execution< actionT >, [22](#)
- start
 - untangle::async::execution< actionT >, [22](#)
- stop
 - untangle::async::execution< actionT >, [23](#)
- untangle, [15](#)
- untangle::async::attachment, [17](#)
- untangle::async::execution< actionT >, [17](#)
 - ~execution, [20](#)
 - action_execute, [25](#)
 - action_is_running, [26](#)
 - action_stop, [25](#)
 - add_action, [23](#)
 - attach, [24](#)
 - bind_action_and_function, [21](#)
 - bind_action_and_method, [20](#)
 - create_instance, [20](#)
 - detach, [24](#)
 - execution, [19](#)
 - is_busy, [22](#)
 - is_running, [21](#)
 - on_finished, [26](#)
 - results, [25](#)
 - run, [22](#)
 - start, [22](#)
 - stop, [23](#)
- untangle::async::execution_poll, [26](#)
 - add, [27](#)
 - get, [28](#)
 - is_running, [28](#)
 - remove, [27](#)
- untangle::async::invalid_attachment, [28](#)
 - invalid_attachment, [28](#)
 - what, [29](#)
- what
 - untangle::async::invalid_attachment, [29](#)